

PH eReferral v0.1 — Specification Holes

Branch: worktree-spec-holes-analysis

2026-06-17

Contents

1	PH eReferral v0.1 — Specification Holes and Counter-Intuitive Sequences	2
1.1	1. Introduction	2
1.2	2. The Intended State Machine	2
1.3	3. Category A — State Machine Transition Gaps	3
1.3.1	A-01 — State Skipping (Zombie Referral)	3
1.3.2	A-02 — Backward Traversal (Temporal Reversal)	5
1.3.3	A-03 — Ghost Draft State	6
1.3.4	A-04 — Orphaned Referred-Onward Path	6
1.3.5	A-05 — Permitted-but-Unmodeled Statuses	8
1.4	4. Category B — Dual-Resource Incoherence	8
1.4.1	B-01 — The Cancellation Gap (Revoked Request, Unaware Task)	8
1.4.2	B-02 — Pre-Delivery Closure (Draft Request, Completed Task)	10
1.4.3	B-03 — The Connectathon Skip (Received State Bypassed in the Test Path)	10
1.5	5. Category C — Semantic Contradictions	10
1.5.1	C-01 — Status vs BusinessStatus Contradiction	10
1.5.2	C-02 — Dual-Rejected Status Collision	11
1.5.3	C-03 — Inconsistent statusReason Representation	12
1.5.4	C-04 — Unresolvable Output Reference (valueCodeableConcept as a Pointer)	12
1.6	6. Category D — Cardinality and Optionality Gaps	13
1.6.1	D-01 — Anonymous Acceptance (Owner-Free Accepted Task)	13
1.6.2	D-02 — Multi-Task Race Condition (Double Acceptance)	13
1.6.3	D-03 — Redundant Invariant	15
1.7	7. Category E — Lifecycle Gaps	15
1.7.1	E-01 — The Back-Referral Black Hole	15
1.7.2	E-02 — Completed vs Closed Semantic Ambiguity	15
1.7.3	E-03 — Infinite Onward Chain	17
1.8	8. Worked Absurd Sequences	18
1.8.1	Sequence 1 — The Zombie Referral	18
1.8.2	Sequence 2 — The Silent Cancellation	19
1.8.3	Sequence 3 — The Double Acceptance	22
1.8.4	Sequence 4 — The Status Schism	22
1.8.5	Sequence 5 — The Eternal Referred-Onward	24
1.8.6	Sequence 6 — The Consultation Summary That Goes Nowhere	24
1.9	9. Cross-Cutting Modelling Issues	26
1.9.1	9.1 The “Required by Narrative, Optional by Profile” Pattern	26
1.9.2	9.2 Obligation Extensions Are Declarations, Not Enforcement	26
1.9.3	9.3 The EReferralReceivingResponse Binding Is Extensible	26
1.9.4	9.4 The Provenance Profile Exists But Is Not Wired Into the Workflow	26
1.10	10. Category F — Operational Scale and Attachment Handling	27
1.10.1	F-01 — No Attachment Size Limit and No Offloading Strategy	27
1.10.2	F-02 — Slow-and-Low Attacks and Cascading Timeout Chains	28

1 PH eReferral v0.1 — Specification Holes and Counter-Intuitive Sequences

Status: Analysis draft — input to CDG and IG authoring team

Branch: worktree-spec-holes-analysis

Date: 2026-06-17

Scope: input/fsh/profiles/ERefTask.fsh, input/fsh/profiles/ERefServiceRequest.fsh, tests/state-model/referral-workflow-state-machine.yaml (upstream branch), input/pagecontent/referral-workflow.md, input/pagecontent/connectathon-readiness.md

1.1 1. Introduction

The PH eReferral v0.1 specification describes a referral workflow using two FHIR resources — ServiceRequest (the clinical order) and Task (the workflow tracker) — coordinated by a state machine with states draft → requested → received → accepted / rejected / referred-onward → completed.

The specification describes this intended path clearly in prose and in the machine-readable YAML (tests/state-model/referral-workflow-state-machine.yaml). However, the FHIR profiles do not mechanically enforce the state machine. The invariants, cardinality constraints, and value bindings in the profiles permit a much larger set of resource states than the workflow narrative intends.

This document catalogues the gap between what is intended and what is permitted. Each hole is a scenario that:

1. Is **rejected by the state machine model** (it would not be a valid TestScript path in the fhir-frog generated suite), but
2. **Passes FHIR profile validation** (SUSHI compiles without error; a FHIR validator accepts the resources).

This gap is not a criticism of the choice to use FHIR Task. It is a normal feature of FHIR — profiles constrain structure and terminology, not process order — and the right response is to understand it explicitly and decide which gaps to close with invariants, which to address with narrative guidance, and which to accept as implementation-time choices.

1.2 2. The Intended State Machine

The state machine from tests/state-model/referral-workflow-state-machine.yaml (upstream 80-workflow-state-machine branch) defines:

```
[START] → draft → requested → received → accepted → completed
                                     ↳ rejected → completed
                                     ↳ referred_onward
                                         (no path to completed defined)
```

Three actors operate in swim lanes: - **Initiator** — creates and sends the referral (draft, requested) - **Receiving Service** — logs, evaluates, and responds (received, accepted, rejected, referred_onward) - **Recipient** — completes the clinical work (completed)

Each state asserts specific FHIR element values:

State	Task.status	Task.businessStatus	ServiceRequest.status
draft	draft (opt.)	—	draft
requested	requested	—	active
received	received	#received	active
accepted	accepted	#accepted	active
rejected	rejected	#rejected	active
referred_onward	rejected	#referred-onward	active
completed	completed	#accepted (carried over)	active

The referred_onward row is already notable: it re-uses Task.status = rejected to represent a semantically different outcome. This is the source of hole C-02.

The diagram for the intended state machine is at tests/state-model/referral-workflow-state-machine.puml.

1.3 3. Category A — State Machine Transition Gaps

The ERefTask profile constrains: - Task.focus 1..1 MS (required ServiceRequest reference)
- Task.requester 1..1 MS (required requester) - Task.intent = #order (fixed) - Invariant
ereferral-task-status-valid (severity #warning) checking the terminal value

None of these constrain **the sequence of status values**. A FHIR server can PUT any valid status value at any time, and a conformant resource can be created in any status.

1.3.1 A-01 — State Skipping (Zombie Referral)

What happens: A Task is created directly in status = completed, or updated from status = requested to status = completed without ever passing through received, accepted, or any intermediate state.

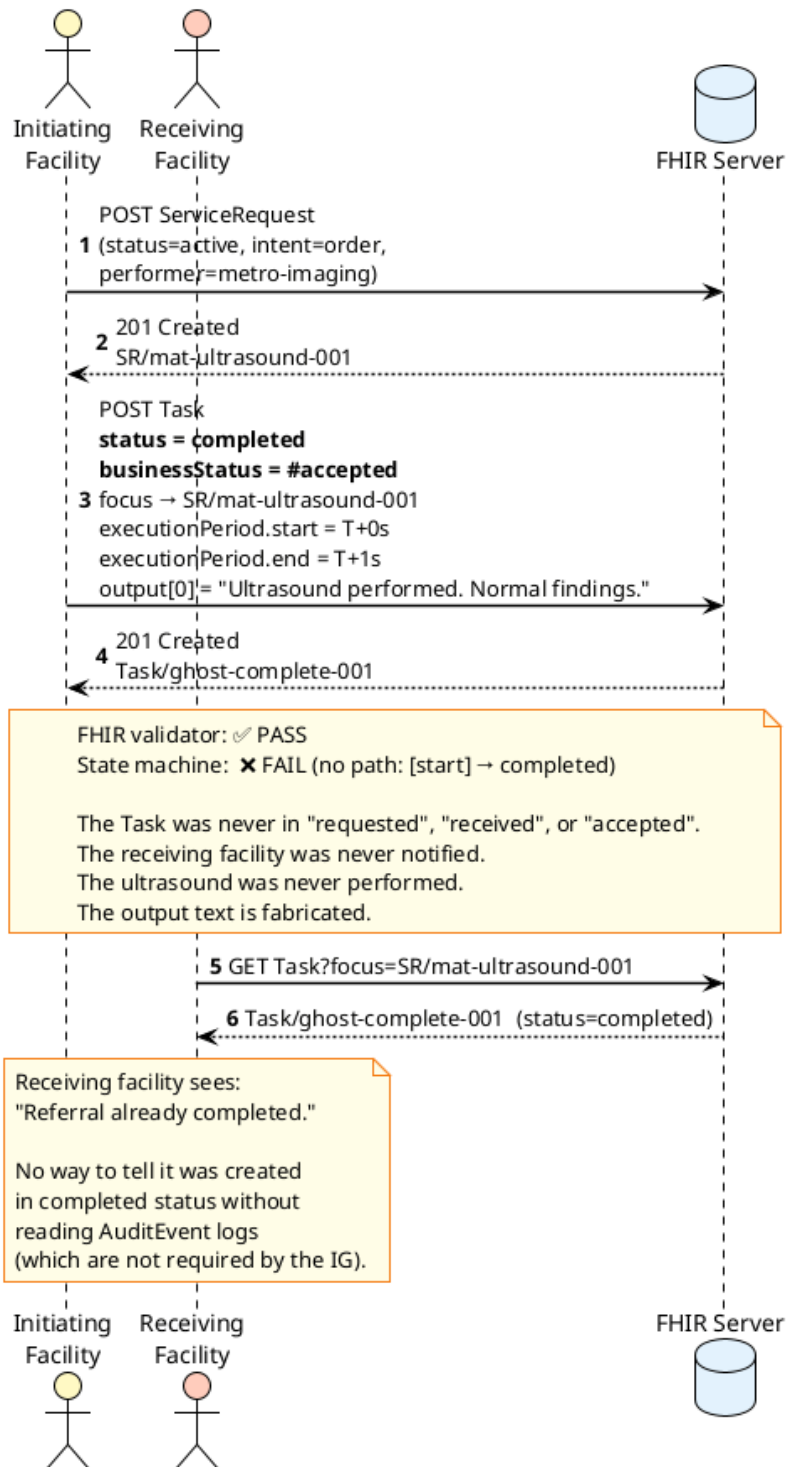
Why the profile permits it: The ereferral-task-status-valid invariant only checks that the final status value is in the permitted set. It does not check what the previous value was. FHIR has no built-in constraint syntax for “previous state.”

Real-world consequence: A referral management system can close a referral as “completed” the moment it is sent, creating a false audit trail that shows the receiving facility accepted and completed the work when it never touched the case. This would be invisible to any conformance validator.

See: analysis/diagrams/A01-zombie-referral.puml

A-01 — Zombie Referral

Task jumps directly to completed, bypassing all intermediate states



, analysis/fsh/AbsurdSequences.fsh instance Absurd-A01-ZombieTask

Minimum fix: An invariant that checks the Task history is not possible in FHIR FHIRPath alone. The correct response is a FHIR \$validate operation with custom rules implemented in the receiving server, or a documented expectation that receiving servers reject status transitions that skip

mandatory intermediate states.

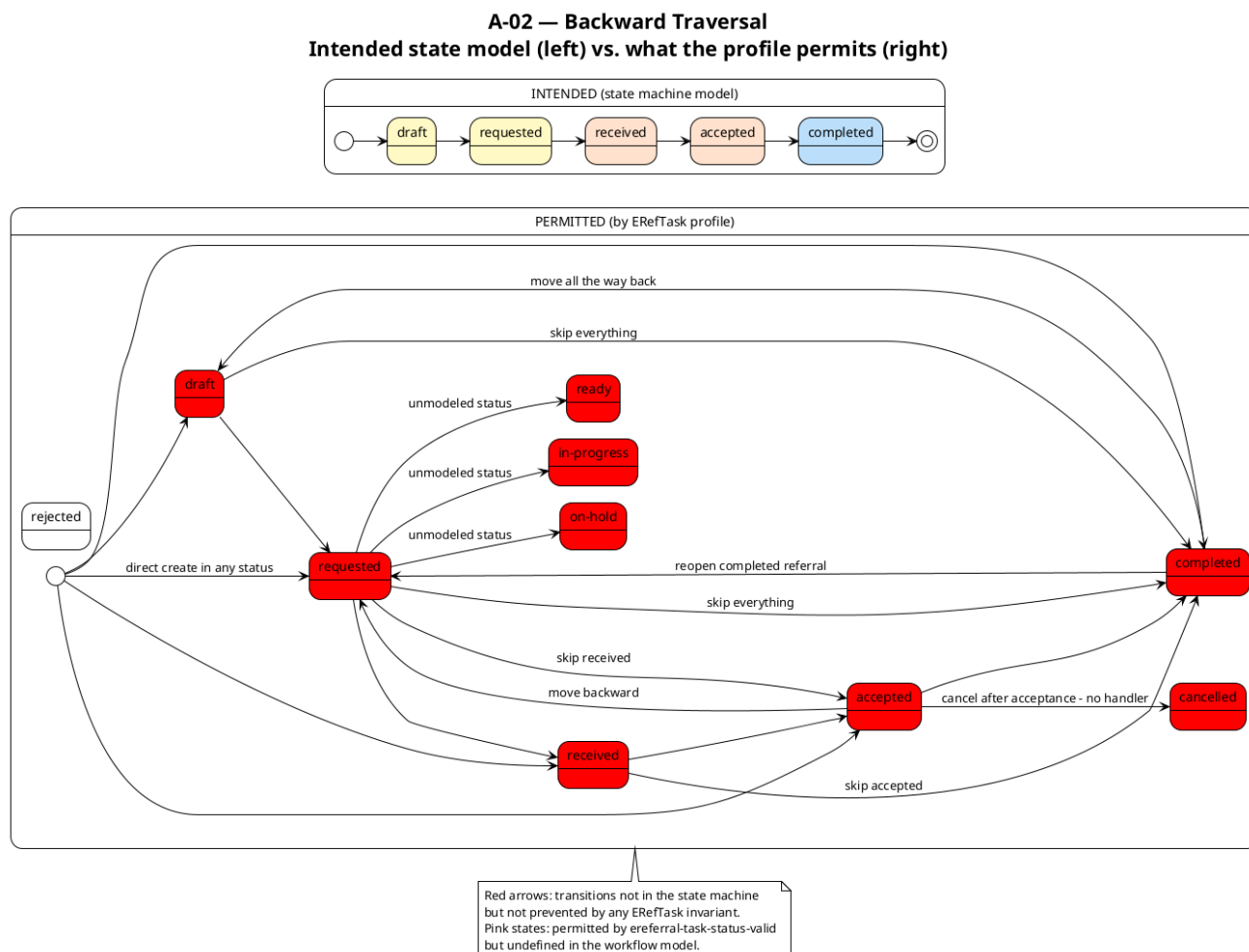
1.3.2 A-02 — Backward Traversal (Temporal Reversal)

What happens: A Task is updated from status = completed back to status = requested. The referral workflow moves backward in time.

Why the profile permits it: The `erefferral-task-status-valid` invariant is severity `#warning`, not `#error`. Even if it were `#error`, it only validates the point-in-time value, not the direction of change. FHIR base spec guidance says Task status should only move forward, but this is not a normative constraint.

Real-world consequence: An adversarial or buggy system can “unclose” a referral, invalidating the audit trail. An accepted referral can be “un-accepted” after clinical work has begun. A billing system relying on the completed status to trigger payment processing could be exploited.

See: `analysis/diagrams/A02-backward-traversal.puml`



, `analysis/fsh/AbsurdSequences.fsh` instance `Absurd-A02-BackwardTask`

1.3.3 A-03 — Ghost Draft State

What happens: The state machine defines a draft state for the Task with an optional Task resource (“Task may not exist yet at draft stage”). The `send_referral` transition then *creates a new Task* in requested status. This means Task never actually passes through draft. The draft state belongs to the ServiceRequest, not to the Task.

Why this matters: A test system using the state machine YAML would generate a TestScript that traverses `create_draft` (creates SR with `status = draft`) followed by `send_referral` (creates Task with `status = requested`). There is no tested path for a Task with `status = draft`. Yet the `ereferred-task-status-valid` invariant explicitly permits draft as a Task status. If an implementer creates a Task in draft status, the profiles accept it but the workflow model has no definition for what it means.

Corollary: The ERefTask profile has no example in `input/fsh/examples/` showing a Task in draft status. Every example starts at requested. If a vendor creates a Task in draft status (perhaps as a “save locally before sending” feature), the IG gives no guidance on when or how to transition it to requested.

Real-world consequence: Two implementations may handle draft Tasks differently: one may never create them; another may create them and leave them in draft indefinitely. A Task in draft status found on a shared server is ambiguous — is it being prepared, or is it a bug?

1.3.4 A-04 — Orphaned Referred-Onward Path

What happens: The `onward_path` in the state machine YAML explicitly ends at `referred_onward` without a completed state:

```
onward_path:
  states:
    - draft
    - requested
    - received
    - referred_onward
  # No 'completed' here
  notes: >
    The referred-onward path ends without a completed state because the
    downstream onward referral is a new workflow instance not modelled here.
```

The original Task — the one with `businessStatus = referred-onward` — stays in `status = rejected` indefinitely. There is no `complete_referred_onward` transition.

Real-world consequence: The original referral Task accumulates in “open” status forever. A receiving facility’s work queue, a dashboard showing “pending referrals,” or a reporting query over `Task.status != completed` will include all referred-onward tasks as perpetually open. If a facility handles 100 referred-onward cases per month and never closes them, after a year they have 1200 “open” tasks cluttering every report.

See: `analysis/diagrams/A04-orphaned-onward.puml`

Fix required: A `referred_onward → completed` transition needs to be defined, with clear semantics: does it close when the onward referral is accepted? When the patient arrives at the onward facility? When the initiating facility confirms?

A-04 — Orphaned Referred-Onward Path

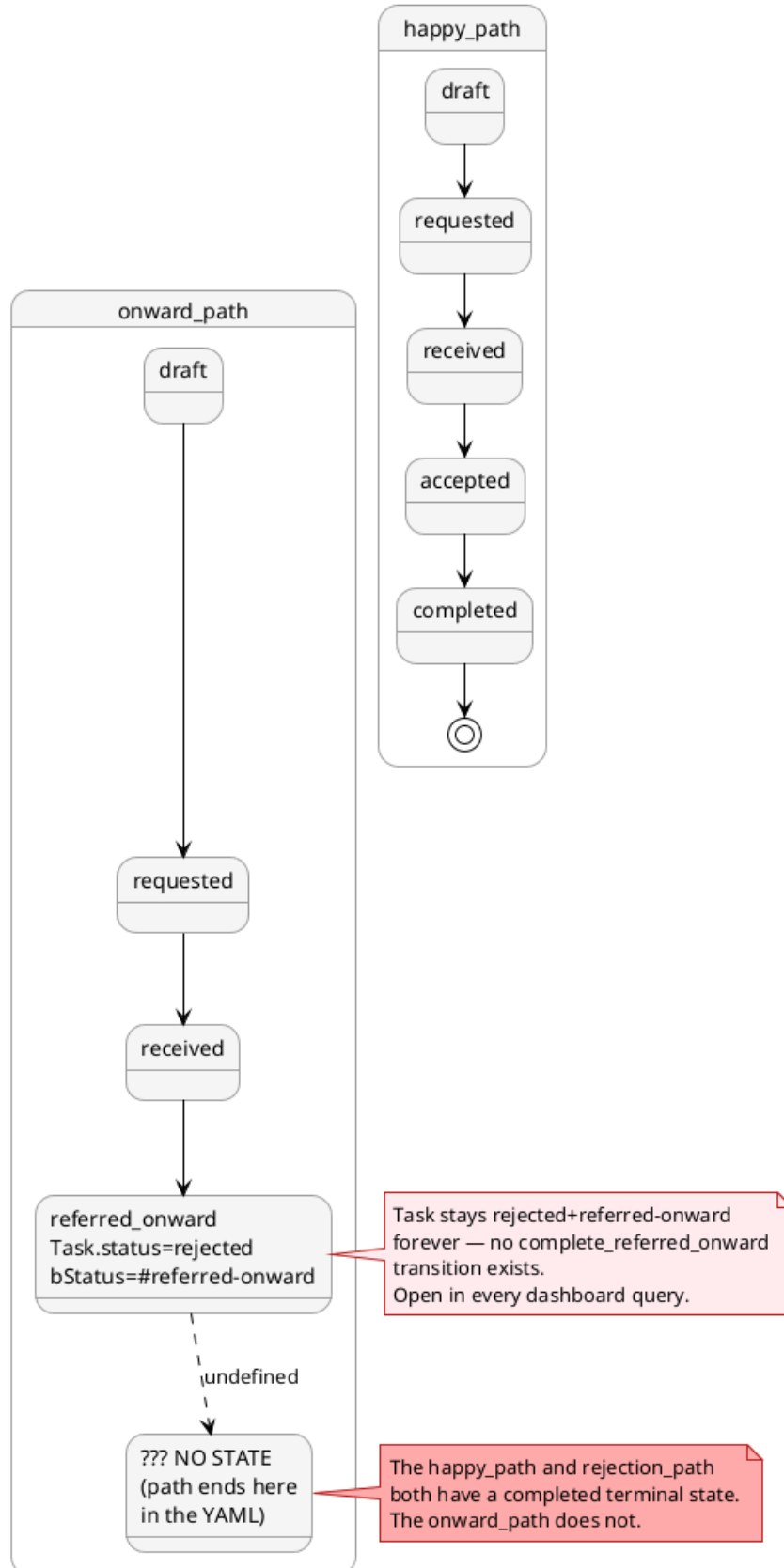


Figure 1: A04-orphaned-onward

1.3.5 A-05 — Permitted-but-Unmodeled Statuses

What happens: The `ereferral-task-status-valid` invariant permits `ready`, `in-progress`, `on-hold`, and `cancelled` as valid Task statuses. None of these appear in the workflow state machine.

Specifically, the invariant expression is:

```
status in ('draft' | 'requested' | 'received' | 'accepted' | 'rejected' |  
          'ready' | 'in-progress' | 'on-hold' | 'completed' | 'cancelled')
```

The model defines: `draft`, `requested`, `received`, `accepted`, `rejected`, `completed` (and `referred_onward` which maps to `rejected`).

Gap: `ready`, `in-progress`, `on-hold`, `cancelled` pass the invariant but have no definition in the eReferral workflow model. Conversely, `failed` and `entered-in-error` (valid in base FHIR) produce a `#warning` from the invariant.

Real-world consequence: A system that uses `in-progress` to mean “the recipient is actively seeing the patient” is profile-conformant but would not be understood by any other implementation. There is no interoperability for these statuses because they are undefined in the IG.

1.4 4. Category B — Dual-Resource Incoherence

`ServiceRequest` and `Task` are separate FHIR resources with no cross-resource invariants between them. Each can be updated independently. The only link is `Task.focus` → `ServiceRequest`, and this is a read-only reference — updating the `ServiceRequest` does not notify or constrain the `Task`.

1.4.1 B-01 — The Cancellation Gap (Revoked Request, Unaware Task)

What happens: The initiating facility cancels the referral by updating the `ServiceRequest` to `status = revoked`. The `Task`, already in `status = accepted`, continues to show that the receiving facility has accepted the case. The receiving facility has no way to know the referral was cancelled unless they poll the `ServiceRequest`.

Why the profile permits it: No invariant on `ERefTask` checks the status of its referenced `focus` (`ServiceRequest`). The profile constrains the `Task`’s own fields only.

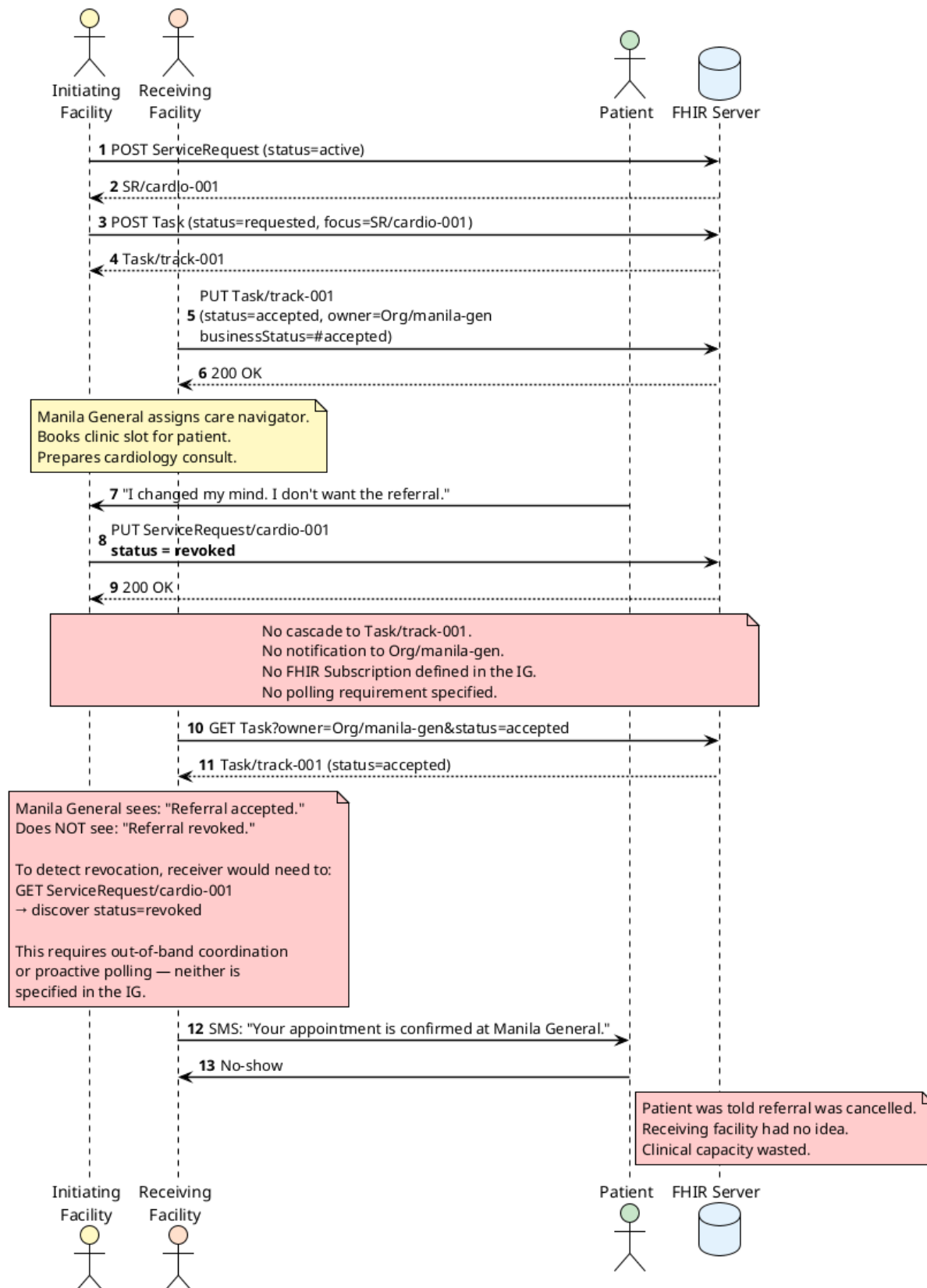
Real-world consequence: The patient changes their mind and the referring physician cancels the referral at 9:00. The receiving facility sees an accepted referral at 9:15 and assigns a care navigator and books a clinic slot. At 11:00 the patient arrives unannounced at a different facility. The receiving facility has wasted capacity; the patient has wasted a journey if they appear at the original receiving facility; no one knows the referral was cancelled until both sides talk.

In a paper system, the referring facility would phone to cancel. FHIR push notifications (Subscription) are not specified in this IG.

See: `analysis/diagrams/B01-cancellation-gap.puml`

B-01 — The Cancellation Gap

ServiceRequest is revoked; the Task owner does not know



1.4.2 B-02 — Pre-Delivery Closure (Draft Request, Completed Task)

What happens: A Task in status = completed references a ServiceRequest in status = draft. The workflow shows “referral completed” for a request that was never sent.

Why the profile permits it: Same reason as B-01 — no cross-resource invariant. ERefTask validates its own content; it does not validate the lifecycle state of the referenced ServiceRequest.

Real-world consequence: This is less likely in a well-behaved system but would occur in a system that creates a Task and ServiceRequest in separate transactions and crashes or is restarted between them, or in a system that processes state updates out of order (e.g., due to message queuing). The result is a “completed” referral where the underlying clinical order never reached the receiving facility.

See: analysis/fsh/AbsurdSequences.fsh instance Absurd-B02-DraftRequestCompletedTask

1.4.3 B-03 — The Connectathon Skip (Received State Bypassed in the Test Path)

What happens: The connectathon readiness page (connectathon-readiness.md) defines the minimum by-hand test as:

Step	Action	Fixture
1	Create referral	Example ServiceRequest
2	Send referral	Task - Requested State
3	Receive referral	Task - Accepted State
4	Respond with outcome	Task - Accepted State
5	Close referral	Task - Completed State

Step 3 says “Receive referral” but links to the **Accepted** fixture, not a **Received** fixture. The received state that appears in the state machine YAML and in the workflow narrative is entirely absent from the connectathon test path.

The coverage map confirms this: > “A separate received state is pending review.”

Real-world consequence: The received state has no worked example and is not exercised in the connectathon test. An implementer following the connectathon guide would never create a Task in received status, rendering that state dead code in every early implementation. After a connectathon, “received” exists in the vocabulary but no system demonstrates it, making interoperability of that state untested.

1.5 5. Category C — Semantic Contradictions

These holes allow a single conformant FHIR resource to carry internally contradictory information.

1.5.1 C-01 — Status vs BusinessStatus Contradiction

What happens: A Task is created with status = accepted and businessStatus = EReferral-WorkflowCS#rejected. FHIR status says the referral is accepted; the eReferral business status says it is rejected.

Why the profile permits it: Both fields are validated in isolation. The profile binds `businessStatus` to the `EReferralReceivingResponse` value set with extensible binding (the code must come from the value set if one fits, but there is no cross-field validation). There is no invariant of the form `Task.status = 'accepted' implies Task.businessStatus = 'accepted'`.

Real-world consequence: A receiving facility system that only reads `Task.businessStatus` concludes the referral was rejected. The initiating facility system that only reads `Task.status` concludes it was accepted. Both are reading a conformant resource. The two systems have silently agreed on opposite outcomes.

See: `analysis/fsh/AbsurdSequences.fsh` instance `Absurd-C01-ContradictoryStatus`

1.5.2 C-02 — Dual-Rejected Status Collision

What happens: Both rejected and referred-onward responses use `Task.status = rejected`. The only way to distinguish them is `Task.businessStatus`.

State machine model:

```
rejected:      Task.status = rejected + businessStatus = #rejected
referred_onward: Task.status = rejected + businessStatus = #referred-onward
```

A FHIR query `GET Task?status=rejected` returns **both** rejected and referred-onward tasks. Any system that filters on status alone conflates “this facility refused and gave up” with “this facility redirected the patient to another facility.”

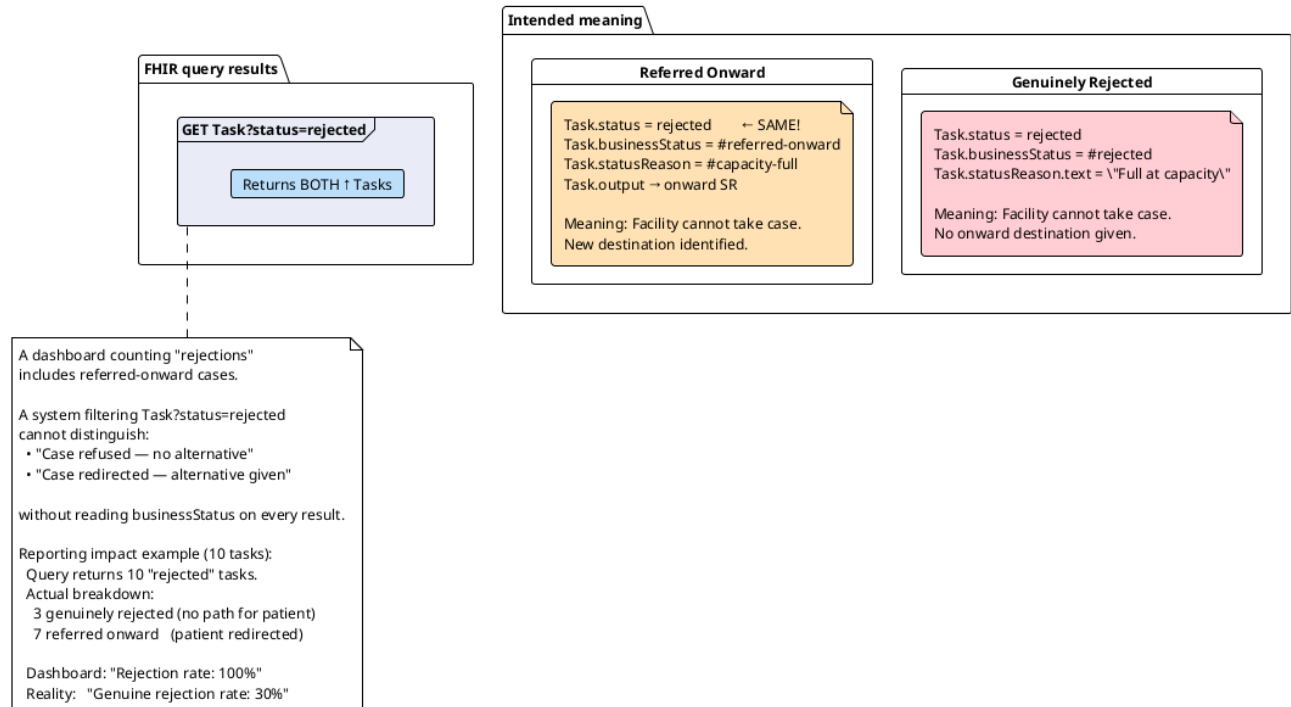
Why this design exists: FHIR base `Task.status` has no code for “referred onward.” The closest is rejected. The `businessStatus` field carries the semantic distinction. This is a reasonable FHIR modelling choice but it creates a usability trap.

Real-world consequence: - Dashboard: “Rejected referrals today: 15” (actually 8 rejected + 7 referred onward) - Report: “Rejection rate this month: 30%” (conflates two clinically different outcomes) - Search: `Task?status=rejected&_include=Task:focus` returns referred-onward tasks; the initiating facility system treating all as “rejected” may stop trying to find an onward destination for a patient who actually needs one.

See: `analysis/diagrams/C02-dual-rejected-status.puml`

C-02 — Dual-Rejected Status Collision

Two distinct outcomes share the same Task.status value



, analysis/fsh/AbsurdSequences.fsh instances Absurd-C02a and Absurd-C02b

1.5.3 C-03 — Inconsistent statusReason Representation

What happens: The two examples in ERefTaskExamples.fsh use statusReason differently:

- ExampleERefTaskRejected: statusReason.text = "Receiving facility cannot take the case..." (free text only, no coding)
- ExampleERefTaskReferredOnward: statusReason = EReferralWorkflowCS#capacity-full (coded, no .text override)

The profile marks statusReason as 0..1 MS with no binding and no structuring invariant. Both representations are conformant. An implementation querying Task.statusReason.coding.code = 'capacity-full' would miss the rejected example; an implementation reading .statusReason.text would get the code display text for the referred-onward example.

Real-world consequence: Two receiving facility systems implement statusReason differently and produce incompatible data. A dashboard that counts capacity-full rejections by coding would undercount. A system that extracts the text reason for communication back to the patient would produce "capacity-full" as a text string for one system.

1.5.4 C-04 — Unresolvable Output Reference (valueCodeableConcept as a Pointer)

What happens: In ExampleERefTaskReferredOnward, the onward referral is referenced via:

```
* output[0].type = EReferralWorkflowCS#onward-referral-request
* output[=].valueCodeableConcept = EReferralWorkflowCS#onward-referral-request
* output[=].valueCodeableConcept.text = "Onward ServiceRequest created: ExampleERefServiceRequest0"
```

The output value is a `CodeableConcept` with the onward `ServiceRequest` identified only by name in the `.text` field. This is a string, not a FHIR Reference. No FHIR client can follow this to retrieve the onward `ServiceRequest`.

Why this matters: `Task.output.value[x]` is polymorphic. `valueReference` would allow a real FHIR reference that a client could resolve. Using `valueCodeableConcept` with a text string instead of a `valueReference` means the onward `ServiceRequest` is identified only by a human-readable name that has no guaranteed uniqueness and cannot be dereferenced by software.

Real-world consequence: A care navigator system that reads `Task.output` to find the onward `ServiceRequest` and prepare a handover packet cannot do so programmatically. It must parse the `.text` string, search for a `ServiceRequest` by name (not a standard FHIR search), and handle the case where the name does not match any resource on the server. The entire referred-onward follow-up chain breaks at this point.

See: `analysis/fsh/AbsurdSequences.fsh` instance `Absurd-C04-UnresolvableOutput`

1.6 6. Category D — Cardinality and Optionality Gaps

1.6.1 D-01 — Anonymous Acceptance (Owner-Free Accepted Task)

What happens: A Task with `status = accepted` and no owner. The profile marks `Task.owner` as `0..1 MS` (optional, must support). Nothing requires owner to be populated when the Task is accepted.

The state machine says the received state should have `Task.owner = receiving facility` (in the PlantUML diagram), and the YAML assertion for received includes checking owner assignment. But this assertion is on the *received* state; there is no equivalent on *accepted*. And there is no profile invariant.

Real-world consequence: Who accepted this referral? If `Task.owner` is absent on an accepted Task, no system can determine which facility took responsibility. A care navigator assigned to follow up on accepted referrals cannot contact the receiving facility. An audit query “which facilities accepted referrals this month” returns empty for this task. If the patient asks “where should I go?”, there is no answer in the data.

See: `analysis/fsh/AbsurdSequences.fsh` instance `Absurd-D01-AnonymouslyAccepted`

1.6.2 D-02 — Multi-Task Race Condition (Double Acceptance)

What happens: Two different receiving facilities each create an accepted Task referencing the same `ServiceRequest`. Both Tasks are profile-conformant. There is no uniqueness constraint on `Task.focus` across Tasks.

Why the profile permits it: FHIR Task has no “one active Task per `ServiceRequest`” constraint. The `ERefTask` profile adds none. A POST of a second Task referencing the same `ServiceRequest` is a valid operation.

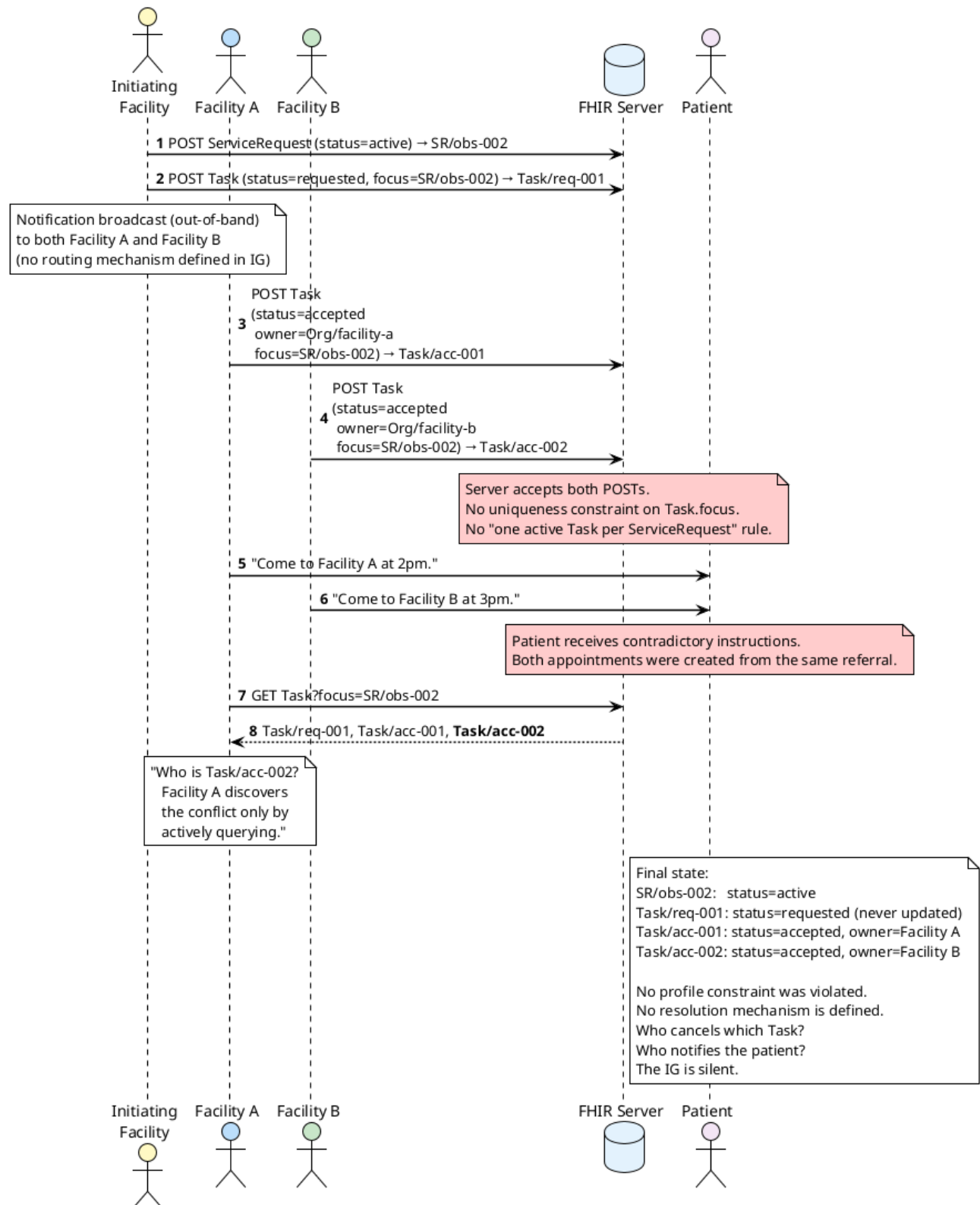
Real-world consequence: This is not a theoretical edge case in a network where multiple receiving facilities receive the same referral notification (e.g., broadcast or mis-routed messages). Both Facility A and Facility B see the referral notification, both create a Task and update it to accepted. The patient is now expected at two facilities. Both facilities have allocated clinical capacity. Neither facility knows about the other’s Task unless they explicitly search.

The resolution mechanism is undefined. How does the initiating facility know which acceptance to honour? How does the “losing” facility close its Task? How is the patient notified?

See: `analysis/diagrams/D02-multi-task-race.puml`

D-02 — Multi-Task Race Condition

Two facilities simultaneously accept the same referral



, analysis/fsh/AbsurdSequences.fsh instances Absurd-D02a and Absurd-D02b

1.6.3 D-03 — Redundant Invariant

What happens: The ERefTask profile declares:

Invariant: `ereferral-task-has-request`

Description: "Task must reference a ServiceRequest (enforced by focus 1..1)"

Severity: #error

Expression: "`focus.exists()`"

The same file documents in a comment: `//` (enforced by focus 1..1, but this provides a clear error message and FHIRPath validation). The cardinality focus 1..1 MS already guarantees `focus.exists()` — a resource that violates the cardinality constraint would be rejected before the invariant is evaluated.

Why this matters: The invariant is not a specification hole (it does no harm), but it creates a false sense of added protection. It will appear in QA output and consume reader attention. More importantly, it models the pattern of “add an invariant as documentation” — if this pattern is extended to the genuinely useful validations (e.g., “owner must exist when status is accepted”), the existing redundant invariant normalises the style but the useful invariants are missing.

1.7 7. Category E — Lifecycle Gaps

1.7.1 E-01 — The Back-Referral Black Hole

What happens: When a referral reaches `status = completed`, the consultation summary is recorded in `Task.output`. But whose Task is it? The Task was created by the receiving service (per the state machine: `receive_referral` is actor `receiving_service`). The initiating facility has no Task in the model that gets updated to carry the consultation summary back to them.

The workflow narrative page says: “Close the workflow when the referral outcome is known. In the current examples, closure is demonstrated by marking the task as completed, recording the end time, and capturing the resulting information.”

But “capturing the resulting information” on the Task does not deliver it to the initiating facility. The Task exists on the shared server (or the receiving facility’s server). If the initiating facility is not polling, it never learns the outcome.

What is missing: A back-referral or back-notification mechanism. This could be: - A second Task (direction: `receiving` → `initiating`) that carries the summary - A FHIR Subscription that notifies the initiating system when the Task completes - A Communication resource from the receiving facility to the referring practitioner - A defined polling expectation

The workflow narrative explicitly defers back-referral: “Back-referral is not modeled as a required end-to-end workflow in this MVP page.” This is an honest disclosure. The consequence is that the v0.1 specification models the first half of a conversation and leaves the reply undefined.

See: `analysis/diagrams/E01-back-referral-gap.puml`

1.7.2 E-02 — Completed vs Closed Semantic Ambiguity

What happens: The state machine defines one terminal state for all outcome types: `completed`. Both the “successful referral” path (`accepted` → `completed`) and the “unsuccessful referral” path (`rejected` → `completed`) end in `completed`.

E-01 — The Back-Referral Black Hole Consultation summary stranded on the receiving side

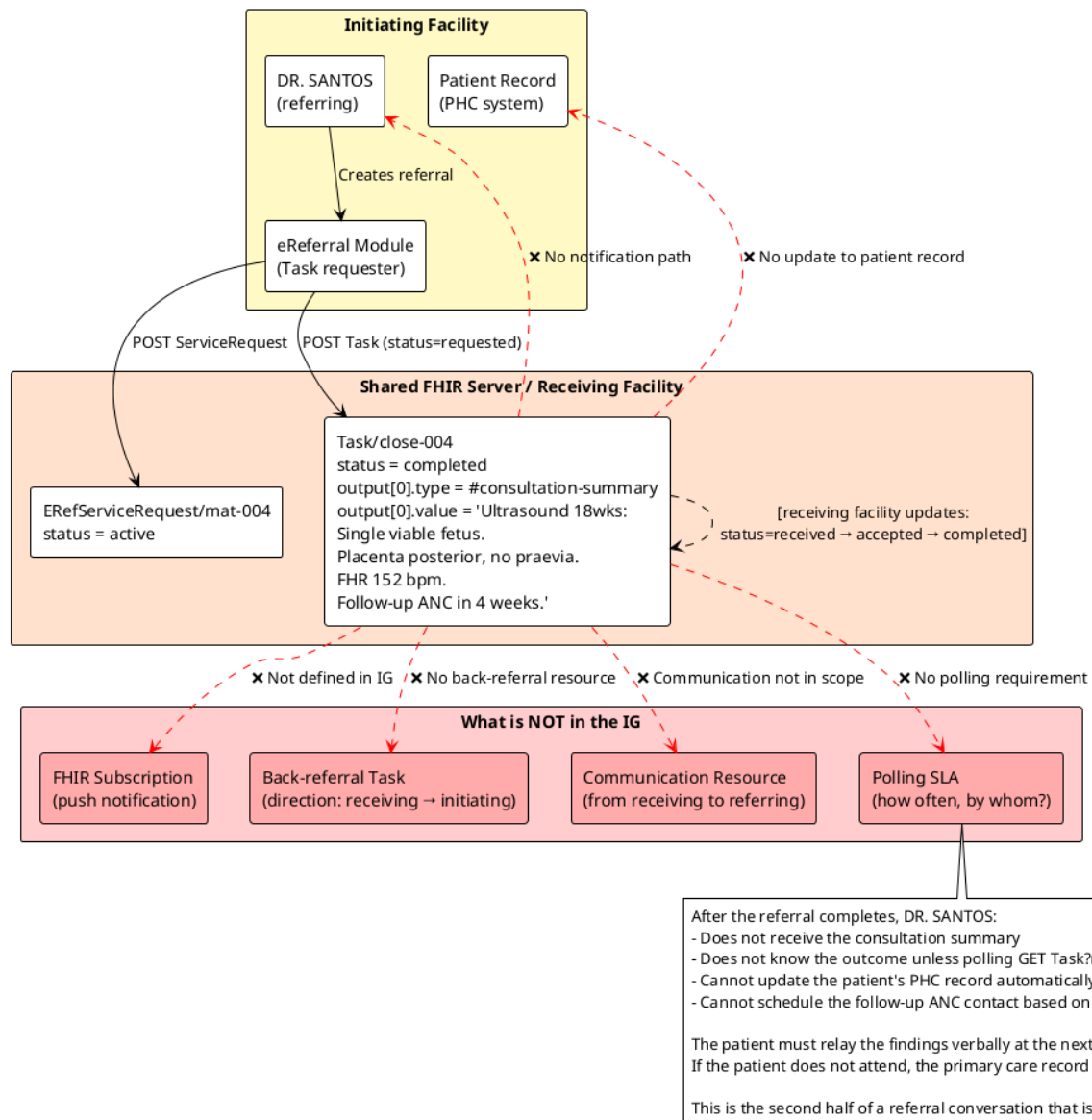


Figure 2: E01-back-referral-gap

In FHIR, `Task.status = completed` conventionally means “the task was accomplished successfully.” `Task.status = cancelled` conventionally means “the task was abandoned or terminated before accomplishment.”

A referral that was rejected and then closed should arguably be cancelled, not completed. Using completed for administrative closure of a rejected case means “completed” appears in dashboards as a success metric when the patient was not served.

Illustration: - 20 referrals this month: 15 accepted, 5 rejected - All 5 rejected → completed via `complete_rejected` - Dashboard: “20 completed referrals this month” ← misleading

The state machine has a `complete_rejected` transition but FHIR’s own cancelled status better represents the outcome for a closed rejection.

See: `analysis/fsh/AbsurdSequences.fsh` instance `Absurd-E02-CompletedRejection`

1.7.3 E-03 — Infinite Onward Chain

What happens: `ERefServiceRequest.replaces` is defined as a recursive reference:

* replaces only `Reference(ERefServiceRequest)`

There is no `maxOccurs` on the chain depth. A sequence of: `> SR#1 → Task#1 (referred-onward) → > SR#2 (replaces SR#1) → Task#2 (referred-onward) → > SR#3 (replaces SR#2) → Task#3 (referred-onward) → ...`

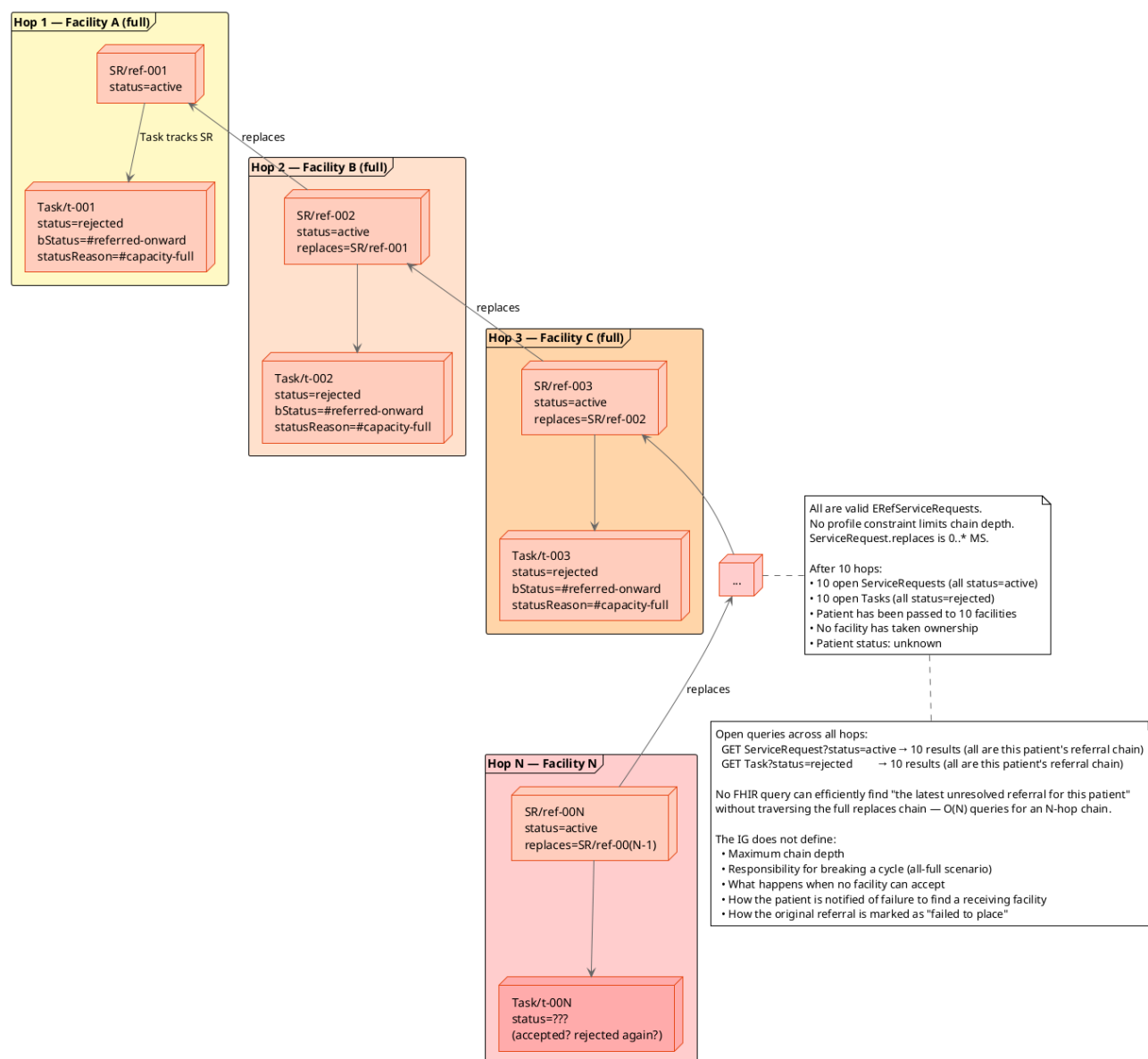
is fully conformant. Each link is a valid `ERefServiceRequest`. There is no invariant limiting the chain length, no defined terminal condition for the “referred onward” chain, and no defined responsibility for who should break the cycle.

Real-world consequence: In a health system where 10 facilities are at capacity, a referral can create a chain of 10 “referred onward” hops before either finding a facility with capacity or failing silently. The patient is passed between systems with no one taking ownership. The original `ServiceRequest` and its 9 successors remain `status = active` unless someone explicitly revokes them.

See: `analysis/diagrams/E03-infinite-onward-chain.puml`

E-03 — Infinite Onward Chain

ServiceRequest.replaces allows unlimited depth; no termination condition



, analysis/fsh/AbsurdSequences.fsh instances Absurd-E03a, Absurd-E03b, Absurd-E03c

1.8 8. Worked Absurd Sequences

This section narrates the six most clinically significant sequences in detail. Each sequence is valid FHIR — a conformant validator would accept all resources. Each is rejected by the intended state machine model.

1.8.1 Sequence 1 — The Zombie Referral

A referring practitioner creates and immediately “completes” a referral, producing a false record of care delivery before the receiving facility has received it.

Step 1: POST ServiceRequest

```
status = active, intent = order, priority = urgent
subject = Patient/charity, requester = PractitionerRole/jane-rhu
performer = Organization/metro-imaging
reasonCode = SNOMED#74290002 "Pregnancy"
```

Step 2: POST Task

```
status = completed           ← DIRECTLY completed, never requested
businessStatus = #accepted   ← claims acceptance happened
focus = Reference(ServiceRequest/from-step-1)
requester = PractitionerRole/jane-rhu
owner = Organization/metro-imaging ← receiving facility listed but never notified
executionPeriod.start = "2026-06-17T09:00:00+08:00"
executionPeriod.end   = "2026-06-17T09:01:00+08:00" ← closed 60 seconds later
output[0].type = #consultation-summary
output[=].valueCodeableConcept.text = "Ultrasound performed. Normal findings."
```

Result: FHIR validator → ☐ PASS

State machine → ☐ FAIL (no transition from [start] to completed)

Clinical truth → The ultrasound was never performed.

Detector: No detector exists in the current profiles. **Potential fix:** Require that a Task in completed status was previously in accepted status, verified by a Provenance resource or an AuditEvent log. Or: require that the executionPeriod.end - executionPeriod.start is non-zero and that the Task was last modified by the receiving facility's system.

PlantUML: See analysis/diagrams/A01-zombie-referral.puml

1.8.2 Sequence 2 — The Silent Cancellation

A referring facility cancels a referral after the receiving facility has accepted it. The receiving facility's Task shows the accepted referral indefinitely.

Step 1: POST ServiceRequest status=active → SR/cardio-001

Step 2: POST Task status=requested → Task/track-001, focus=SR/cardio-001

Step 3: PUT Task/track-001 status=accepted, businessStatus=#accepted, owner=Org/manila-gen

← Receiving facility accepts. Manila General assigns care navigator.

Step 4: PUT ServiceRequest/cardio-001 status=revoked

← Referring physician: "Patient deceased, referral no longer needed."

State of data after Step 4:

ServiceRequest/cardio-001: status = revoked ← cancelled

Task/track-001: status = accepted ← still accepted!

Manila General's care coordinator queries:

GET Task?owner=Org/manila-gen&status=accepted

→ Returns Task/track-001 ← No indication the referral was cancelled.

Result: Manila General prepares capacity for a patient who will never arrive. The Task must be manually inspected (\$follow Task.focus → ServiceRequest) to discover the revocation. No notification occurs.

PlantUML: See analysis/diagrams/B01-cancellation-gap.puml

A-01 — Zombie Referral

Task jumps directly to completed, bypassing all intermediate states

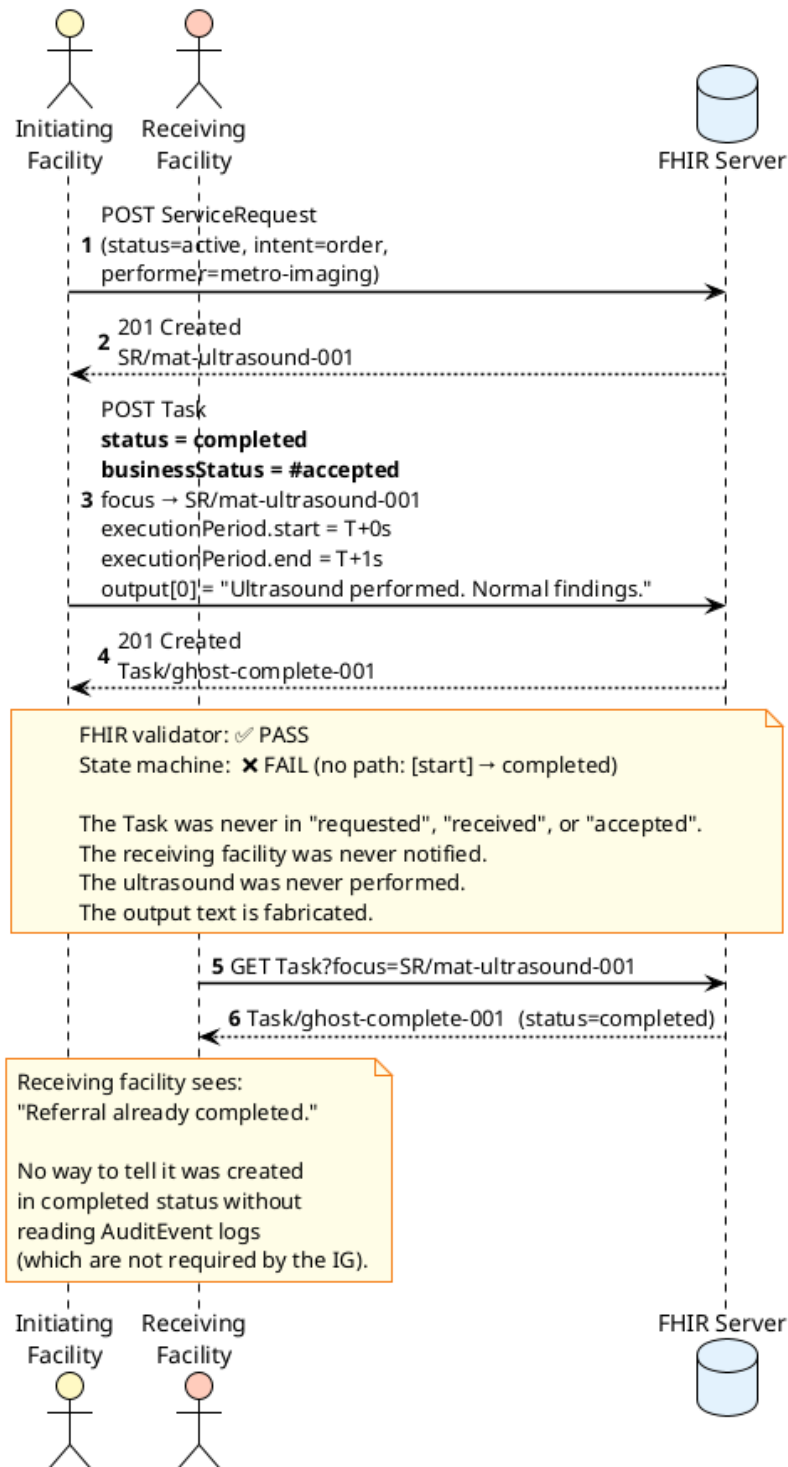


Figure 3: A01-zombie-referral

B-01 — The Cancellation Gap

ServiceRequest is revoked; the Task owner does not know

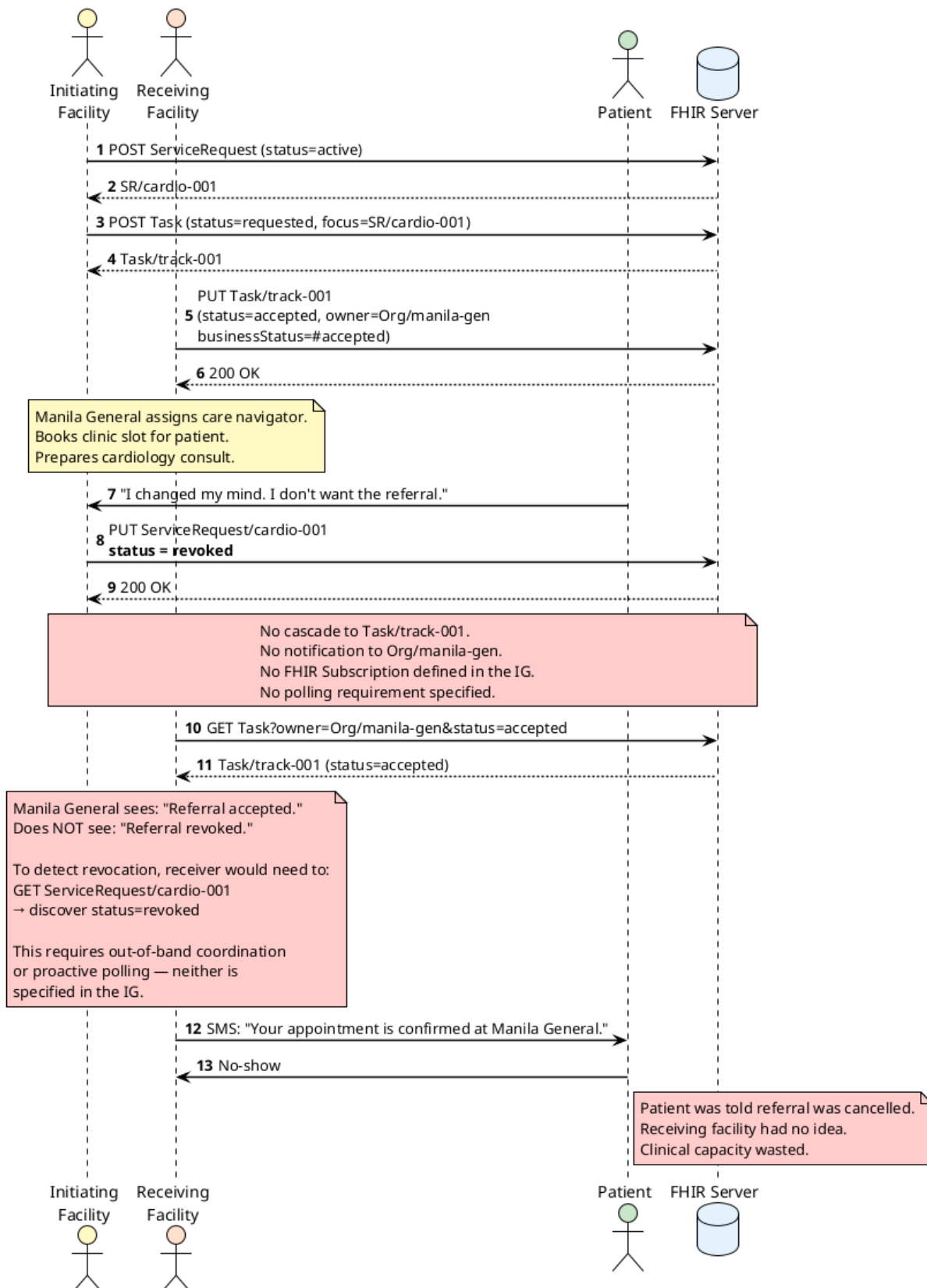


Figure 4: B01-cancellation-gap

1.8.3 Sequence 3 — The Double Acceptance

Two receiving facilities simultaneously accept the same referral. The patient is expected at both; neither facility knows about the other.

Step 1: POST ServiceRequest status=active → SR/obs-002

Step 2: POST Task status=requested, focus=SR/obs-002 → Task/req-001

[Referral notification broadcast to two facilities]

Step 3a (Facility A): POST Task

status=accepted, businessStatus=#accepted

focus=Reference(SR/obs-002), owner=Reference(Org/facility-a)

→ Task/acc-001

Step 3b (Facility B): POST Task

status=accepted, businessStatus=#accepted

focus=Reference(SR/obs-002), owner=Reference(Org/facility-b)

→ Task/acc-002

State of data:

SR/obs-002: status=active

Task/req-001: status=requested ← original tracking task unchanged

Task/acc-001: status=accepted ← Facility A claims acceptance

Task/acc-002: status=accepted ← Facility B claims acceptance

FHIR query: GET Task?focus=SR/obs-002 → returns 3 Tasks, all active

Result: Both facilities validate as conformant. The patient is booked at both. Neither system knows about the race. When the patient arrives at Facility A, Facility B's task remains open. No resolution mechanism is defined.

PlantUML: See analysis/diagrams/D02-multi-task-race.puml

1.8.4 Sequence 4 — The Status Schism

The Task carries contradictory status information: the FHIR canonical status says one thing; the eReferral businessStatus says the opposite.

POST Task:

status = #accepted ← FHIR canonical: "case accepted"

businessStatus = EReferralWorkflowCS#rejected ← business layer: "case rejected"

intent = #order

focus = Reference(SR/chest-pain-003)

requester = Reference(PractitionerRole/rhu-santos)

owner = Reference(Org/manila-general)

Result: FHIR validator → ☐ PASS (binding is extensible; no cross-field invariant)

System A (reads Task.status):

GET Task/status-schism → "accepted" → care navigator assigned → patient told to proceed

System B (reads Task.businessStatus):

GET Task/status-schism → "rejected" → patient told referral was declined → new referral created

Result: Same FHIR resource, two diametrically opposite clinical decisions, both technically correct reads of the resource as defined by the profile.

D-02 — Multi-Task Race Condition

Two facilities simultaneously accept the same referral

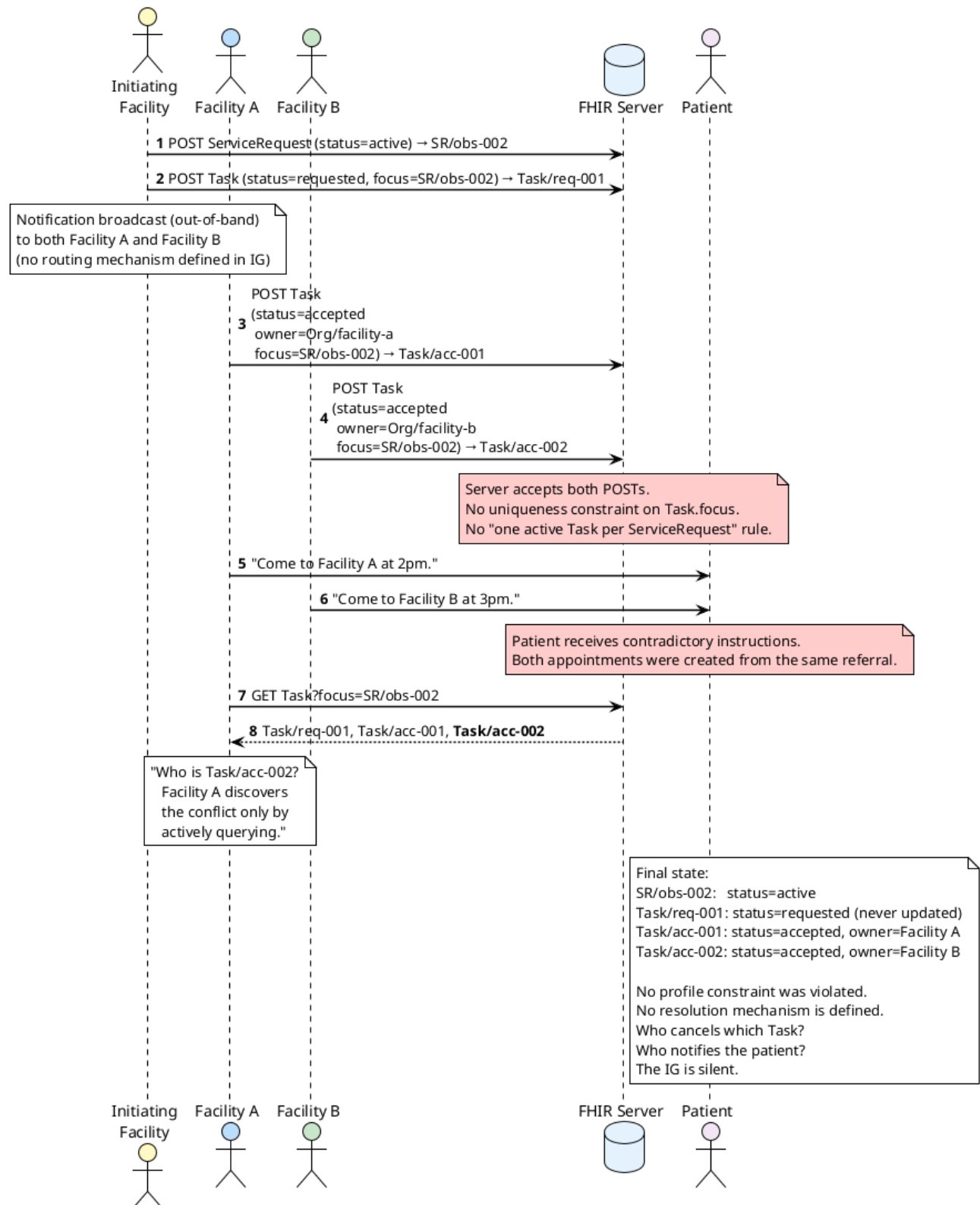


Figure 5: D02-multi-task-race

1.8.5 Sequence 5 — The Eternal Referred-Onward

A referred-onward case generates a chain of ServiceRequests that never terminates. All tasks remain open. No facility takes ownership.

```
POST SR/ref-001 status=active
POST Task/t-001 status=requested focus=SR/ref-001
PUT Task/t-001 status=rejected businessStatus=#referred-onward → Facility A full
POST SR/ref-002 status=active replaces=SR/ref-001
POST Task/t-002 status=requested focus=SR/ref-002
PUT Task/t-002 status=rejected businessStatus=#referred-onward → Facility B full
POST SR/ref-003 status=active replaces=SR/ref-002
POST Task/t-003 status=requested focus=SR/ref-003
PUT Task/t-003 status=rejected businessStatus=#referred-onward → Facility C full
... [repeats indefinitely]
```

Open tasks after 10 hops:

Task/t-001 through Task/t-010: all status=rejected, businessStatus=#referred-onward
SR/ref-001 through SR/ref-010: all status=active

None of these resources are in a terminal state. All appear as active referrals. No facility has taken ownership. The patient is waiting.

PlantUML: See analysis/diagrams/E03-infinite-onward-chain.puml

1.8.6 Sequence 6 — The Consultation Summary That Goes Nowhere

A referral completes with a consultation summary on the Task. The initiating facility never receives it because there is no notification or back-referral mechanism.

[Happy path: SR/mat-004 requested → received → accepted → completed]

```
PUT Task/close-004:
  status = completed
  executionPeriod.end = "2026-06-20T14:30:00+08:00"
  output[0].type = #consultation-summary
  output[=].valueCodeableConcept.text = "Ultrasound at 18 weeks: single viable fetus.
    Placenta posterior, no praevia. FHR 152 bpm. Follow-up ANC in 4 weeks recommended."
```

State of data:

Task/close-004.status = completed ← on receiving facility's server
DR. SANTOS (initiating practitioner) → knows nothing.

DR. SANTOS queries:

```
GET Task?requester=PractitionerRole/jane-rhu&status=completed
→ Returns Task/close-004 only if:
  (a) Task is on a shared server, and
  (b) DR. SANTOS' system is polling, and
  (c) The system parses valueCodeableConcept.text for clinical content.
→ Returns nothing if Task is on receiving facility's server.
```

Result: The consultation summary exists but is stranded on the receiving side. The patient's primary care record is never updated. The referring practitioner cannot follow up because they do not know the outcome. The patient must recall and report the findings themselves.

E-03 — Infinite Onward Chain

ServiceRequest.replaces allows unlimited depth; no termination condition

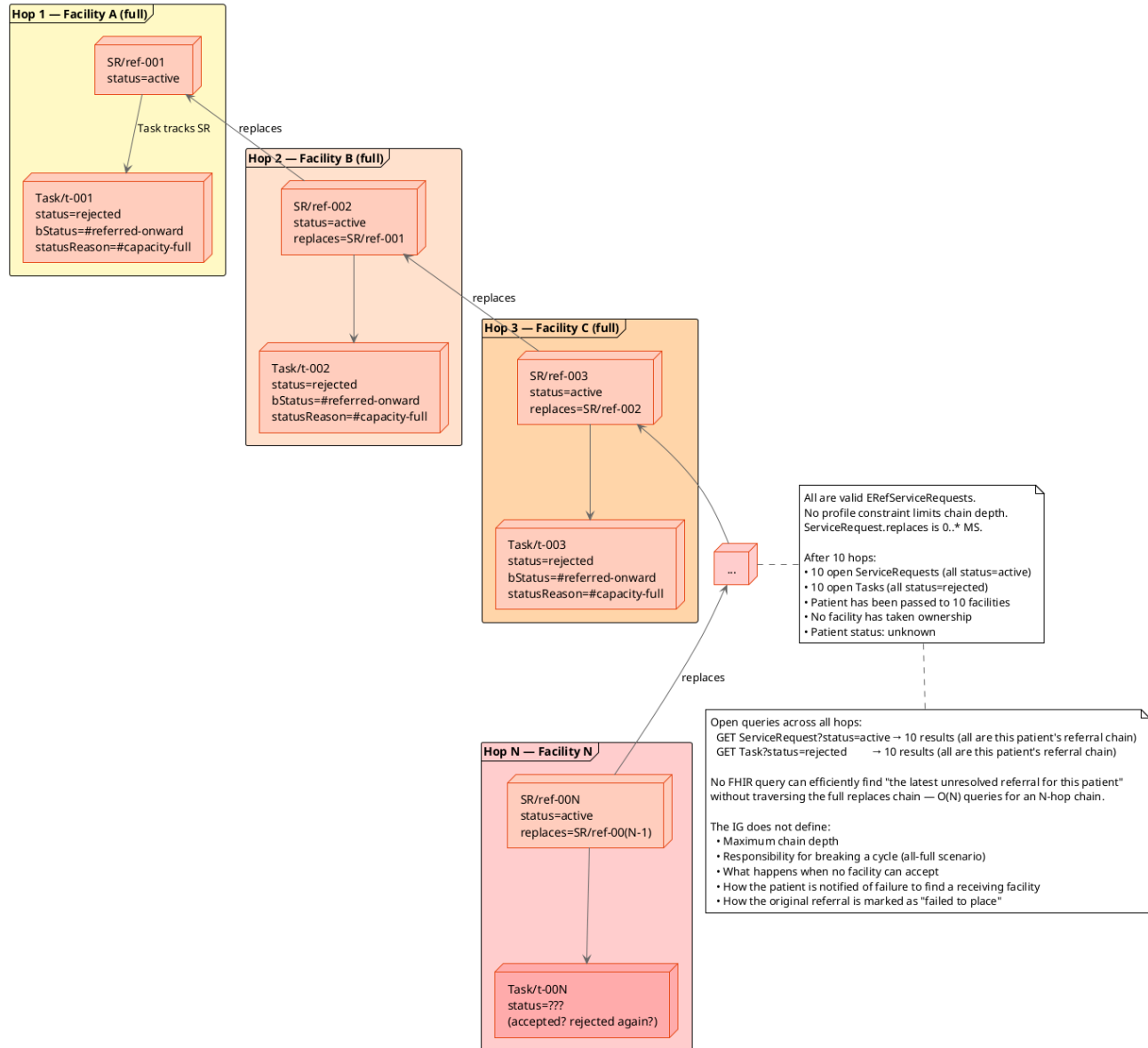


Figure 6: E03-infinite-onward-chain

1.9 9. Cross-Cutting Modelling Issues

Beyond individual holes, several structural decisions affect the entire specification.

1.9.1 9.1 The “Required by Narrative, Optional by Profile” Pattern

Many fields that the workflow narrative treats as essential are marked 0..1 MS (optional, must support) in the profiles. The `ObligationOptional` rule set is applied consistently, meaning:

- `Task.authoredOn` (0..1 MS) — narrative step 7 mentions “noting when it was last updated”
- `Task.lastModified` (0..1 MS) — no timestamp means no audit trail
- `ServiceRequest.authoredOn` (0..1 MS) — “Date of Referral” is TDG REF-13
- `ServiceRequest.subject` (0..1 MS) — “Patient Full Name” is TDG REF-15/21

A patient reference on the `ServiceRequest` is optional according to the profile constraint. A referral with no patient reference is conformant.

The reason for this pattern is stated in `AGENTS.md`: “constrain only what is clinically necessary” (Profile Minimalism). But there is a tension between minimalism and the workflow’s operational requirements. A referral that does not identify the patient, the referring date, or the receiving facility may be minimal from a FHIR standpoint but is clinically non-functional.

1.9.2 9.2 Obligation Extensions Are Declarations, Not Enforcement

The profiles use `ObligationRequired` and `ObligationOptional` rule sets from `input/fsh/ruleSets/obligation`. These generate obligation extensions on the `StructureDefinition`, which describe what actors SHOULD do. They do not produce FHIR invariants. A FHIR validator does not enforce obligations — only a custom obligation-aware validator would.

This means the `ObligationRequired` annotation on `Task.focus` and `Task.requester` communicates intent but does not mechanically enforce it beyond what the cardinality already does.

1.9.3 9.3 The `EReferralReceivingResponse` Binding Is Extensible

`Task.businessStatus` is bound to the `EReferralReceivingResponse` value set with extensible binding. This means any code that is not in the value set but fits a more specific system can be used. In practice, extensible means “required unless you have a good reason to use something else.”

For a small closed workflow with four defined response states, an extensible binding is permissive. If the binding were required, the `status/businessStatus` contradiction in C-01 would still exist (both fields would be individually bound to their respective code sets), but at least the `businessStatus` vocabulary would be closed.

1.9.4 9.4 The Provenance Profile Exists But Is Not Wired Into the Workflow

`ErefProvenance` is defined and linked via `ServiceRequest.relevantHistory`. The workflow narrative mentions “professional signature” and “date of signature” as TDG requirements. But the state machine has no transition that requires or creates a Provenance resource. No invariant requires `ServiceRequest.relevantHistory` to be populated. The audit trail is optional and unlinked to the state machine.

1.10 10. Category F — Operational Scale and Attachment Handling

This category is distinct from the state machine holes: it does not relate to profile conformance or state transitions, but to the operational assumptions baked into the specification's resource model and the real-world consequences of deploying it at national scale in the Philippines.

1.10.1 F-01 — No Attachment Size Limit and No Offloading Strategy

What happens: The `ERefServiceRequest` profile includes `supportingInfo` as `0..*` MS, constrained to reference clinical resources. In practice, clinical summaries include scanned paper forms, lab result PDFs, imaging reports, and referral slips. FHIR allows `Attachment.data` to carry base64-encoded binary content inline within the resource. The IG sets no size limit on attachments and defines no offloading strategy.

What the IG provides: `ServiceRequest.supportingInfo` references `PHCoreCondition` | `PHCoreObservation` | `PHCoreProcedure` | `PHCoreMedicationAdministration` | `PHCoreImmunization`. These are structured resources, not documents. However, in practice: - `PHCoreObservation` may include `Observation.component.valueAttachment` for image data - `DocumentReference` (not yet profiled in this IG) would carry PDFs - Binary resources with base64 content are standard FHIR practice for scanned documents

The IG is silent on whether and how attachments should be handled, which means each implementation will decide independently, producing incompatible approaches.

Philippines national scale — calculation:

The Philippines public health system includes approximately: - 21,000+ Barangay Health Stations (BHS) - 2,600 Rural Health Units (RHUs) - ~1,900 public hospitals (district, city, provincial, tertiary) - ~9,000 PhilHealth-accredited private facilities

Total: ~34,500 health facilities capable of generating referrals.

A conservative active-referral scenario (early national rollout):

Parameter	Conservative	Moderate	Full rollout
Active eReferral facilities	5,000	15,000	30,000
Referrals per facility per day	10	25	40
Attachments per referral	2	3	5
Average attachment size	300 KB	500 KB	700 KB
Data per day	3 GB	56 GB	420 GB
Data per year	1.1 TB	20 TB	153 TB

At full rollout with 40 referrals/day/facility and 5×500 KB PDFs each: - 30,000 facilities $\times$ 40 $\times$ 5 $\times$ 500 KB = **3 TB/day** - Per year: **~1.1 PB**

The attack / stress vector:

A single malformed (or simply misconfigured) referral submission with 40×1 MB inline attachments produces a 40 MB JSON POST body. HAPI FHIR has no normative default body size limit — it inherits from the web container (Tomcat, typically capped at a few MB by default, but over-rideable). If the server has been configured for large payloads (as would be needed for real clinical use), a single client submitting 1,000 such requests would ingest 40 GB of data. This is an unintentional denial-of-service vector, not a deliberate attack.

What the standard does not provide:

FHIR R4 base specification has no normative attachment size limit. The IG offers no guidance on:

1. Maximum attachment size per resource or per transaction

2. Whether Attachment.data (inline binary) or Attachment.url (external pointer) should be used
3. An IHE MHD (Mobile Health Documents) pattern for external document storage
4. A CDN, object storage (S3/Azure Blob/GCS), or national document repository integration
5. Deduplication of identical attachments across referrals (the same lab result may be attached to 10 referrals for the same patient)
6. Retention and deletion policy
7. Server-side CapabilityStatement declaration of maximum request body size
8. Compression requirements

The absence of these specifications means each implementation will: - Choose different storage backends - Set different size limits (or none) - Implement incompatible deduplication strategies - Produce servers with wildly different performance profiles

See: analysis/diagrams/F01-attachment-scale.puml

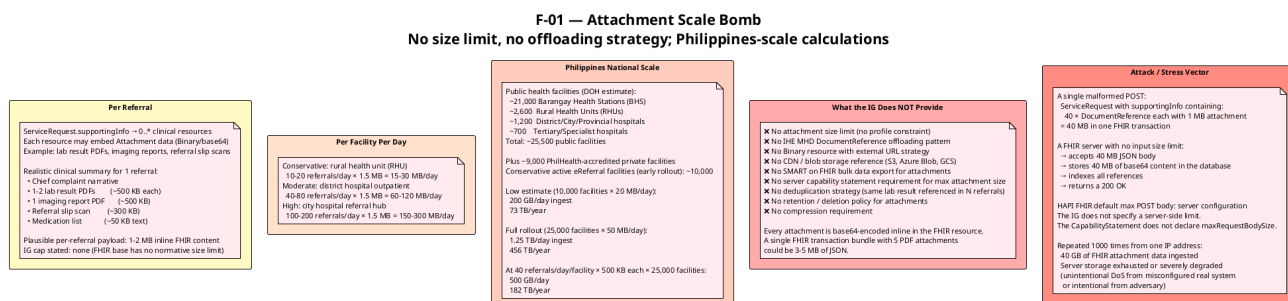


Figure 7: F01-attachment-scale

Minimum fix: The IG should normatively state whether attachments are in-scope for v0.1 and, if so, whether inline (Attachment.data) or external (Attachment.url) attachment patterns are required. At minimum, a SHALL-level statement capping inline attachment size (e.g., “Attachments included via Attachment.data MUST NOT exceed 100 KB; larger documents MUST use Attachment.url referencing an external storage endpoint”) would prevent the unbounded storage problem. A reference to IHE MHD for production attachment handling would complete the picture.

1.10.2 F-02 — Slow-and-Low Attacks and Cascading Timeout Chains

This hole is a direct consequence of F-01. Because the IG places no cap on payload size and no guidance on server-side connection handling, large referral payloads create a family of denial-of-service vectors that are difficult to distinguish from legitimate behaviour on a rural Philippine network.

1.10.2.1 What slow-and-low means in this context A “slow and low” attack keeps server resources occupied for as long as possible using the minimum amount of traffic needed to avoid triggering rate-based defences. The classic form (Slowloris) sends HTTP request headers one byte at a time to hold a TCP connection open without triggering a read-body timeout. Against a FHIR server with large-payload support, there is a more effective variant: **slow body upload**.

The attacker — or simply a legitimate facility on a poor rural connection — sends the HTTP headers at full speed (enough to pass any connection-rate limiter or WAF), then trickles the body at a rate just fast enough to prevent the server’s read timeout from firing. The server’s thread is locked, waiting for the rest of a payload that may never arrive at a useful rate.

1.10.2.2 Why large FHIR payloads make this severe A normal FHIR resource (Patient, Observation, ServiceRequest with text fields only) might be 3-15 KB. A server read-body timeout

of 60 seconds is generous for that size.

An ERefServiceRequest with inline PDF attachments is a different order of magnitude:

Referral content	Estimated JSON payload
Clinical fields only (no attachments)	~10 KB
+ 1 lab result PDF (300 KB)	~415 KB (base64 overhead $\times 1.37$)
+ 3 PDFs $\times$ 500 KB	~2.1 MB
+ 5 PDFs $\times$ 700 KB (emergency referral with imaging)	~4.8 MB
+ 10 scanned forms $\times$ 1 MB (complex case)	~13.7 MB
+ 40 PDFs $\times$ 500 KB (the scenario from F-01)	~27.4 MB

At a rural FHIR client upload speed of 512 Kbps (typical for Philippine rural 4G on a congested cell):

Payload	Upload time at 512 Kbps
2.1 MB	~33 seconds
4.8 MB	~75 seconds
13.7 MB	~214 seconds (3.6 minutes)
27.4 MB	~428 seconds (7 minutes)

These are not attack durations. They are the expected upload times for a genuine rural health unit on a real Philippine network. The server cannot tell the difference between:

- A legitimate RHU in a poor-coverage barangay uploading a complex referral over 7 minutes
- An attacker deliberately trickling a 27 MB payload to hold a connection open

1.10.2.3 The server resource model during a large upload HAPI FHIR (the reference server in docker-compose.yml) processes requests using a standard Tomcat servlet thread pool. Each incoming HTTP request occupies:

1. **One Tomcat thread** — held from the first byte of the request until the last byte of the response. Default Tomcat max threads: 200.
2. **One JPA database transaction** — opened when the request enters the FHIR write path. Default transaction timeout: none (inherits from the database connection pool timeout, typically not configured for FHIR servers in development).
3. **One database connection** from the HikariCP pool. Default pool size: 10-20.
4. **Heap memory** for deserialising the JSON body. A 27 MB JSON payload inflates to 2-4 $\times$ its size as Java objects during FHIR parsing. A single large upload can hold 54-108 MB of heap.

A server with 200 Tomcat threads and a 7-minute upload time:

$200 \text{ threads} \div 7 \text{ minutes per upload} = \sim 28$ concurrent large uploads saturate the pool

After saturation:

New legitimate requests $\rightarrow$ queued or rejected (503 Service Unavailable)
Database connections exhausted $\rightarrow$ any FHIR read/write times out
GC pressure from heap $\rightarrow$ JVM pauses degrade all response times

This is not a theoretical bound. It is the operating condition when a modest eReferral rollout runs its morning peak across a rural province.

1.10.2.4 The timeout cascade chain When a large upload finally completes (or is interrupted), the timeout chain begins:

1. HTTP read timeout (Tomcat)
 - └ Not configured in docker/hapi/application.yaml
 - └ Defaults to Tomcat connector timeout (~20s for headers, but body read can be indefinite on a persistent connection)
2. FHIR validation timeout
 - └ HAPI runs FHIRPath evaluation over every element in the submitted resource
 - └ 40 DocumentReferences × FHIRPath expressions = $O(N^2)$ evaluation for some checks
 - └ No validation timeout configured in the docker stack
 - └ A 27 MB resource may take 30–120 seconds to validate fully
3. JPA transaction timeout
 - └ HAPI writes the resource + all its contained elements in one transaction
 - └ Writing 27 MB of base64 content to a PostgreSQL BYTEA column takes seconds
 - └ If the transaction times out mid-write, the DB rolls back
 - └ HAPI may not report the rollback cleanly → client receives 500 Internal Server Error
 - └ Client retries → server processes the same 27 MB again
4. Client retry amplification
 - └ Retry 1: same 27 MB, 7-minute upload, transaction rollback
 - └ Retry 2: same again
 - └ 3 retries × 27 MB = 81 MB consumed in total for zero successful writes
5. Base64 decode CPU cost
 - └ 27 MB of base64 text → 20 MB of decoded binary
 - └ Decoded on every request, including retries
 - └ CPU cost is proportional to payload size; concurrent large uploads can saturate a single-core container
6. PostgreSQL TOAST overflow
 - └ HAPI stores the raw resource JSON in a CLOB column (hfj_res_ver.res_text)
 - └ PostgreSQL stores large CLOBs via TOAST (The Oversized Attribute Storage Technique)
 - └ TOAST compresses inline, then spills to an overflow table for very large values
 - └ Each 27 MB resource creates a large TOAST entry that is read back in full on every resource retrieval, including bundle searches
 - └ A search that returns 10 such resources reads 270 MB from PostgreSQL

1.10.2.5 The rural Philippines legitimate-traffic version None of this requires a malicious actor. It is what happens when:

- An emergency occurs in a remote barangay
- The BHS nurse creates a referral with photographs of the patient and a scanned copy of the paper referral slip
- The upload takes 8 minutes over congested LTE
- The server times out at 5 minutes (if a timeout is configured) or holds the thread for 8 minutes (if not)
- 30 BHSs in the same province are doing the same thing simultaneously during morning peak (07:00–09:00 is empirically the busiest period for referrals in Philippine primary care)
- The shared FHIR server for the province is unavailable for everyone during peak hour

This is the “slow and low” attack that is not an attack. The IG’s silence on payload limits, connection handling, and offloading strategy means the default server configuration — which is what every implementer starts from — is vulnerable to it without any adversary involvement.

1.10.2.6 What the IG does not specify

Gap	Consequence
No normative HTTP body size limit	Server accepts arbitrarily large payloads
No <code>server.tomcat.connection-timeout</code> guidance	Threads held indefinitely during slow uploads
No JPA transaction timeout guidance	Database connections exhausted by slow large writes
No recommendation to use Transfer-Encoding: chunked for large uploads	Client must buffer entire payload before sending
No asynchronous (Prefer: <code>respond-async</code>) upload pattern for large bundles	All large uploads are synchronous, blocking threads
No retry-after guidance	Client retries immediately, amplifying the load
No <code>CapabilityStatement</code> <code>maxRequestSize</code> declaration	Clients cannot self-limit before uploading
No attachment offloading (see F-01)	Every attachment traverses the full synchronous path
No CDN or pre-signed URL pattern for binary content	No escape valve for large binary uploads

1.10.2.7 Specific HAPI configuration gaps in `docker/hapi/application.yaml` The current `docker/hapi/application.yaml` (from the repository) configures: - `defer_indexing_for_codesystems_of_size` 100000 (needed for terminology, well-configured) - Remote terminology services (needed, well-configured) - The AGENTS.md documents the UCUM fragment workaround

It does **not** configure:

```
# Missing from docker/hapi/application.yaml:
server:
  tomcat:
    connection-timeout: 30s          # not set – defaults to 20s for headers only
    max-http-form-post-size: 10MB    # not set – unlimited

spring:
  servlet:
    multipart:
      max-file-size: 5MB             # not set
      max-request-size: 10MB         # not set

hapi:
  fhir:
    max_binary_size: 1048576         # not set – defaults to no limit in many builds
    validation:
      requests_enabled: true          # enabled but no timeout for large resources
```

These are not exotic hardening options. They are standard Java web server configuration that every production deployment should make explicit. Their absence in the reference Docker stack means every implementer who uses this stack as a starting point inherits the unprotected configuration.

See: analysis/diagrams/F02-slow-low-attack.puml

1.11 12. Recommendations

These are ranked by clinical risk, not implementation effort.

Priority	Hole	Recommended action
1	B-01 (Cancellation Gap)	Define how Task owners are notified of ServiceRequest cancellation. Either add a FHIR Subscription requirement or specify that the Task must be cancelled when ServiceRequest is revoked.
2	C-02 (Dual-Rejected Collision)	Rename the referred_onward FHIR status or add a new custom code to Task.status via profile extension. Alternatively, rename the state and make the distinction searchable without reading businessStatus.
3	E-01 (Back-Referral Gap)	Define a back-referral communication path. Minimum viable: a second Task from the receiving side addressed to the initiating facility, or a FHIR Communication resource.
4	A-01 / A-02 (State Skipping / Backward Traversal)	Add server-side business rules as narrative guidance (at minimum) or as custom FHIR operation constraints. Document that Task status transitions must be strictly ordered and forward-only as a SHALL statement.
5	C-01 (Status vs BusinessStatus Contradiction)	Add an invariant: when businessStatus is in EReferralReceivingResponse, the invariant should check that Task.status is consistent with the business status code.
6	A-04 (Orphaned Referred-Onward)	Define a complete_referred_onward transition in the state machine. Specify when the original Task should be closed.
7	D-01 (Anonymous Acceptance)	Add an invariant: Task.status = 'accepted' implies Task.owner.exists().
8	C-04 (Unresolvable Output Reference)	Change Task.output.value[x] for #onward-referral-request type to use valueReference pointing to the onward ERefServiceRequest.
9	D-02 (Multi-Task Race)	Document the “one active Task per ServiceRequest” expectation as a server capability requirement. Consider adding a FHIR CapabilityStatement note.
10	E-03 (Infinite Onward Chain)	Add an invariant or narrative guidance limiting the depth of ServiceRequest.replaces chains (e.g., maximum 3 hops).
11	B-03 (Connectathon Skip)	Add a Task - Received State fixture to the connectathon test pack. Link Step 3 to it.

Priority	Hole	Recommended action
12	A-03 (Ghost Draft)	Either add a Task draft example or remove draft from the <code>ereferral-task-status-valid</code> invariant's permitted set.
13	E-02 (Completed vs Closed Semantic)	Use <code>Task.status = cancelled</code> for administrative closure of rejected referrals. Reserve completed for cases where clinical service was delivered.
14	D-03 (Redundant Invariant)	Remove <code>ereferral-task-has-request</code> ; the focus 1..1 MS cardinality already enforces it.
15	9.1 (Required by Narrative, Optional by Profile)	Elevate <code>ServiceRequest.subject</code> , <code>ServiceRequest.authoredOn</code> , and <code>ServiceRequest.performer</code> to 1..1 MS or add invariants. A referral without a patient or destination is not operationally useful.
16	F-01 (Attachment Scale / No Offloading Strategy)	Add a normative cap on inline <code>Attachment.data</code> size (e.g., 100 KB) and require <code>Attachment.url</code> for larger files. Reference IHE MHD for production attachment handling. Add a <code>CapabilityStatement</code> server requirement declaring maximum request body size. At national scale (30,000 facilities, 40 referrals/day, 5 PDFs each), unaddressed attachment storage reaches ~1.1 PB/year.
17	F-02 (Slow-and-Low / Cascading Timeouts)	Add normative server configuration requirements: HTTP body read timeout (30s), JPA transaction timeout (60s), and a <code>CapabilityStatement</code> <code>maxRequestSize</code> declaration. Require <code>Prefer: respond-async</code> support for bundles above a defined threshold. Mandate the <code>Attachment.url</code> offloading pattern (F-01) as the primary remedy — this eliminates the slow-body vector for attachment content entirely. Without this, a standard morning peak across 30 rural BHSs can exhaust a provincial FHIR server's thread pool before any adversarial actor is involved.

F-02 — Slow-and-Low Attack / Cascading Timeout Chain

Large inline attachments + no payload limits + poor rural connectivity

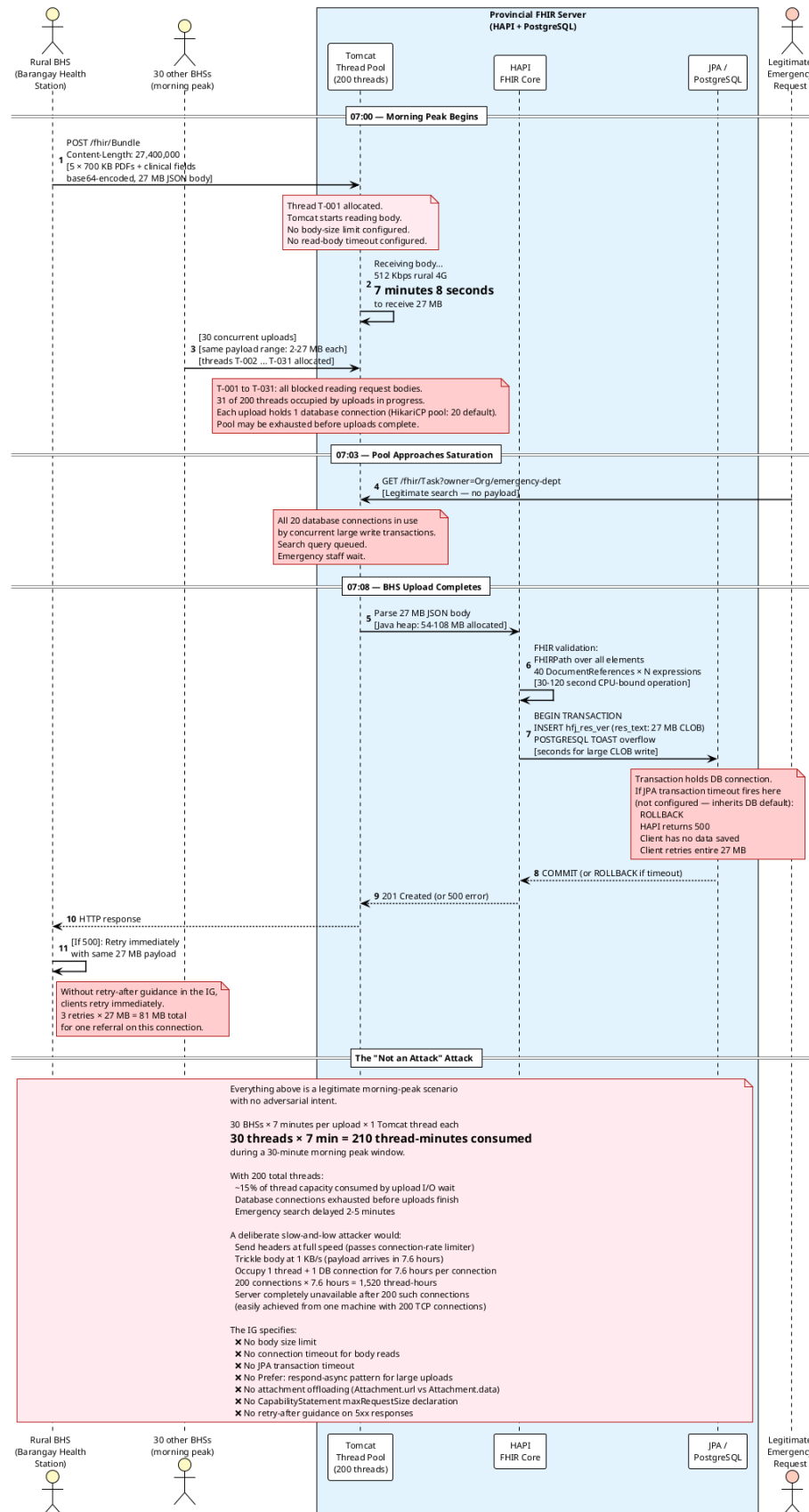


Figure 8: F02-slow-low-attack